Desarrollo de Aplicaciones Web

AJAX

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces

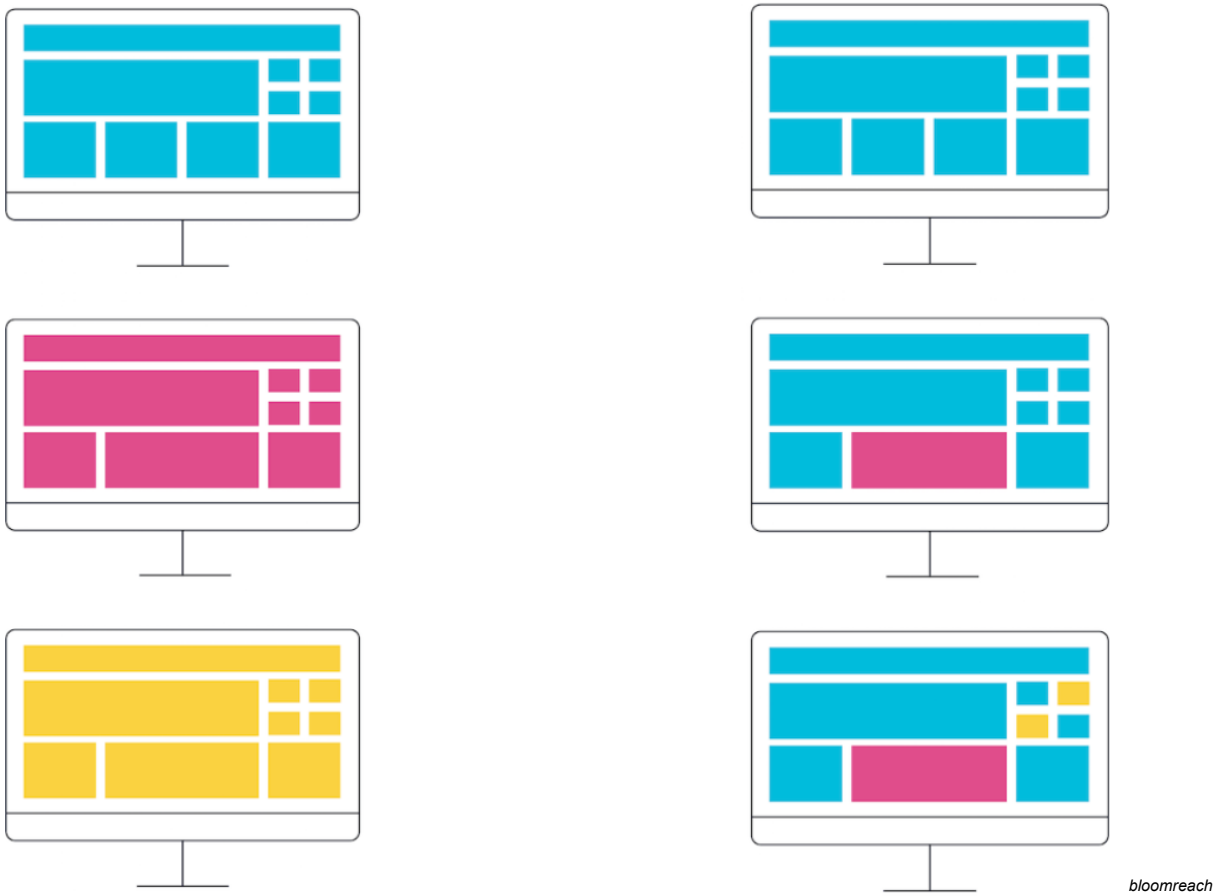
MPA vs SPA

Multi-Page Application (MPA)→ Es el sitio web tradicional.

Cada página representa mostrada es una petición especifica a un servidor que devuelve, tanto en imágenes como en contenido, un fichero HTML completo que se corresponde con la página que hemos pedido.

Single Page Application (SPA)→ Es una única página HTML que de manera dinámica, sobreescribe el contenido de la misma en lugar de cargar una nueva.

MPA vs SPA



MPA **SPA**

JavaScript *vs* AJAX

Procesamiento “síncrono” vs “asíncrono”

Cuando un navegador encuentra una etiqueta <script> en una página la procesa de inmediato, deteniendo la carga del resto de la página → Procesamiento síncrono.

Problema → Si en el <script> hay una parte que requiere la carga de datos externos, el proceso de carga se detiene a la espera de esos datos externos.

AJAX permite realizar cargas de forma asíncrona. Si la información solicitada para cargar no está presente no la procesa de inmediato y continúa con la carga del resto de la página → Procesamiento asíncrono.

JavaScript *vs* AJAX

AJAX.- *Asynchronous JavaScript + XML*

Nace en febrero de 2005 de la mano de *Jesse James Garret*, como consecuencia de un artículo titulado: "*Ajax a new approach to web applications*"

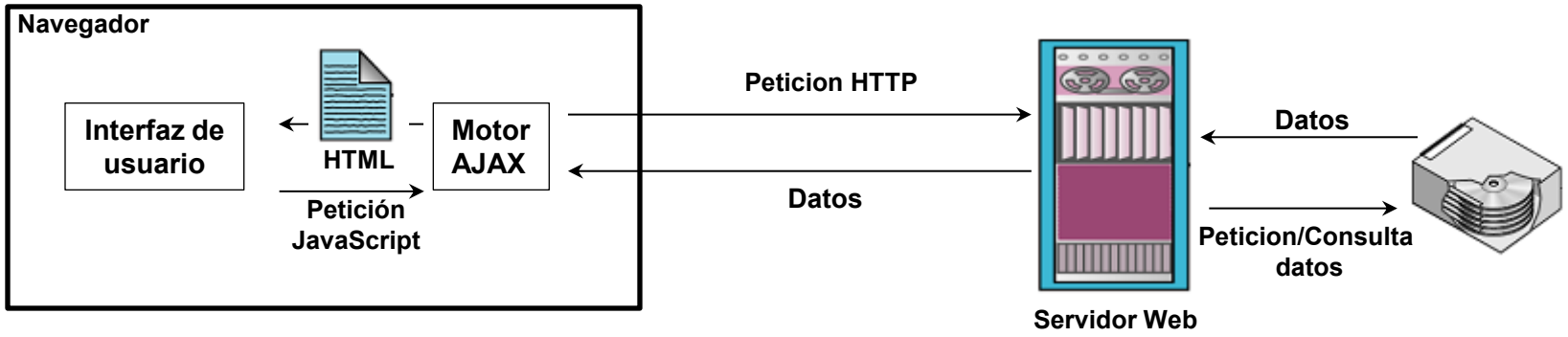
Ajax permite:

- Actualizar una página web, sin recargarla.
- Hacer peticiones de datos a un servidor, después de que la página esté cargada.
- Recibir datos de un servidor, después de que la página esté cargada.
- Enviar datos a un servidor, en *background.*
- Desarrollos SPA.

JavaScript *vs* AJAX

AJAX.- *Asynchronous JavaScript + XML*

Nace en febrero de 2005 de la mano de *Jesse James Garret*, como consecuencia de un artículo titulado: "*Ajax a new approach to web applications*"



Motor AJAX.- objeto o función *JavaScript* que es llamada siempre que se realiza una petición de información al servidor.

JavaScript *vs* AJAX

AJAX.- *Asynchronous JavaScript + XML*

AJAX utiliza una combinación de:

- Un objeto *XMLHttpRequest* construido dentro de un navegador.
- *JavaScript* y *HTML DOM* para mostrar los datos en el navegador.

XMLHttpRequest → objeto javaScript diseñado por *Microsoft* y actualmente estándar de *W3C*. Es utilizado para insertar datos en la página sin recarga de la misma.

Creación:

```
let miXML= new XMLHttpRequest();
```

JavaScript *vs* AJAX

AJAX.- *Asynchronous JavaScript + XML*

El objeto *XMLHttpRequest.*

Métodos:

- abort() → cancela solicitud.
- getAllResponseHeaders() → devuelve informacíón de cabecera
- getResponseHeader() → devuelve informacíón de cabecera específica
- open(method, url, async, user, psw() → especifiica el tipo de solicitud
- send() → envía solicitud al servidor, en modo GET
- send(string) → envía solicitud al servidor en modo POST
- setRequestHeader() → añade un par etiqueta/valor a la cabecera para envío

Atributos:

- onreadystatechange → define una función para ser llamada cuando el readyState cambie
- onload → define una función para ser llamada cuando la petición es recibida
- readyState → almacena el estado de XMLHttpRequest
- responseText → retorna respuesta en formato texto
- responseXML → retorna respuesta en formato XML
- status → retorna el número de estado de la solicitud
- statusText → retorna el estado de la solicitud

XMLHttpRequest

AJAX.- *Asynchronous JavaScript + XML*

El objeto *XMLHttpRequest.*

Atributos:

- onload → define una función *callback* para ser llamada cuando la *petición* sea recibida.
- *onreadystatechange* → define una función *callback* para ser llamada cuando el *readyState* cambie.
- *status* → retorna el número de estado de la solicitud

readyState:

- Petición no inicializada
- Conexión con el servidor, establecida
- Petición recibida
- Procesando petición
- Petición finalizada y respuesta lista

status:

- 200: "OK"
- 403: “Acceso prohibido"
- 404: "Pagina no encontrada"
- ……

XMLHttpRequest

AJAX mediante el objeto *XMLHttpRequest*

Funcionamiento:

- Crear el objeto *XMLHttpRequest* → *let* miHttp= new XMLHttpRequest();
- Definir la función *callback* → miHttp.onreadystatechange= mifunction() { … };
  miHttp.onload= mifunction() { … }
- Abrir el objeto *XMLHttpRequest* → miHttp.open("GET", “miFile");
- Enviar un pettición al servidor → *miHttp.send();*

Nota.- *los navegadores modernos soportan el atributo onload del objeto XMLHttpRequest y por tanto, en la actualidad, se suele utilizar el atributo onload en este tipo de llamadas.*

XMLHttpRequest

AJAX mediante el objeto *XMLHttpRequest*

Crear el objeto XMLHttpRequest

Abrir el objeto XMLHttpRequest

Petición:

```
let x= new XMLHttpRequest();
x.open('GET', 'url_filename', true)
x.send()
```

"true" indica que lo haga de forma asíncrona

Enviar un pettición al servidor

Respuesta:

Ejecutar la función callback definida

```
x.onload= funtion() {
   if (x.status == 200) {
         ______
         ______
   }
}
```

XMLHttpRequest

AJAX mediante el objeto *XMLHttpRequest*

```
<h1>ETSE</h1>
<h2 id="demo"></h2>
```

```
AJAX USC
```

fileTXT.txt

ETSE

AJAX USC

```
cargaTexto();
```

ETSE

Error: 404

```
function cargaTexto() {
    let xhttp = new XMLHttpRequest();
    xhttp.onload = function() {
        if (this.status == 200)
            document.getElementById("demo").innerHTML= this.responseText;
        else
            document.getElementById("demo").innerHTML= "Error: " + this.status;
    };
    xhttp.open("GET", "fileTXT.txt", true);
    xhttp.send();
}
```

XMLHttpRequest

AJAX mediante el objeto *XMLHttpRequest + CallBack*

```
<h1>ETSE</h1>
<h2 id="demo"></h2>
```

```
AJAX USC
```

fileTXT.txt

ETSE

AJAX USC

```
cargaTexto(miVisualizacion)
```

ETSE

Error: 404

```
function cargaTexto(miCBack) {
  let xhttp= new XMLHttpRequest();
  xhttp.onload = function() {
      if (this.status == 200)
          miCBack(this.responseText);
      else
          miCBack("Error: " + this.status);
  };
  xhttp.open("GET", "fileTXT.txt", true);
  xhttp.send();
}
```

```
function miVisualizacion(txt){
  document.getElementById("demo").innerHTML= txt;
}
```

Promesas

Promesa→ Objeto utilizado para representar la terminación o el fracaso en la ejecución de una operación asíncrona.

Sintáxis:

```
let miPromesa= new Promise(function(miExito, miFracaso) {

  miExito(expresión_1); → Función callback que recibe un valor, cuando la
                          operación asíncrona finaliza con éxito.

  miFracaso(expresión_2); → Función callback que recibe la razón por
                            la cual, una operación asíncrona no finaliza con éxito.
});

miPromesa.then(
  function(value) { Código cuando éxito },
  function(error) { Código cuando fracaso }
);
```

Promesas

Promesa→ Objeto utilizado para representar la terminación o el fracaso en la ejecución de una operación asíncrona.

En la práctica, una promesa es un objeto devuelto al cual se adjuntan funciones callback, cuya ejecución depende del éxito o fracaso de la operación asíncrona.

Método *then()* → Método que recibe 2 argumentos consistentes en funciones *callback* relativas al éxito o fracaso de una promesa.

```
p.then(siExito[, siFracaso])
```

En la práctica, la llamada a la función que realiza la acción asíncrona, quedaría:

```
FuncionTareaAsincrona(parametros).then(siExito, siFracaso)
```

Nota *→ Las funciones callback usadas como parámetros del método .then() no son obligatorias.*

Promesas

Promesa→ Objeto utilizado para representar la terminación o el fracaso en la ejecución de una operación asíncrona.

```
<h1>Promesas</h1>
<p></p>
<p></p>
```

```
let miP = new Promise(function(res, rej) {
  let n= Math.random();

  let ok= false;
  if (n>0.5)
    ok= true;

  if (ok)
    res({"msg":"Esto es un OK", "valor": n});
  else
    rej({"msg":"Esto es un Error", "valor": n});
});
```

```
miP.then(
    function(rsp){
        miVz(rsp)
    },
    function(rsp){
        miVz(rsp)
    }
```

```
function miVz(rsp) {
    let el= document.getElementsByTagName("p");
    el[0].innerHTML = rsp.msg;
    el[1].innerHTML = rsp.valor;
}
```

Promesas

Esto es un OK

0.9145268646377406

Promesas

Esto es un Error

0.3099184176700589

XMLHttpRequest

AJAX mediante el objeto *XMLHttpRequest* + *Promesa*

```
<h1>ETSE</h1>
<h2 id="demo"></h2>
```

```
AJAX USC
```

fileTXT.txt

ETSE

AJAX USC

ETSE

Error: 404

```
let miP= new Promise(function(resuelve, rechaza){
  let xhttp = new XMLHttpRequest();
  xhttp.onload = function() {
      if (this.status == 200)
          resuelve(this.responseText);
      else
          rechaza("Error: " + this.status);
  };
  xhttp.open("GET", "fileTXT.txt", true);
  xhttp.send();
});
```

```
miP.then(
  function(v){miVisualizacion(v);},
  function(e){miVisualizacion(e);}
);
```

```
function miVisualizacion(txt){
  document.getElementById("demo").innerHTML= txt;
}
```

XMLHttpRequest

AJAX mediante el objeto *XMLHttpRequest*

```
<h2>ETSE</h2>
<ul></ul>
```

```
function cargaTexto(){
    let xhttp= new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            let xmlDoc= this.responseXML;
            let tagArr= xmlDoc.getElementsByTagName("Rector");

            visualizaXML(tagArr);
        }
    };
    xhttp.open("GET", "uniXMLFile.xml", true);
    xhttp.send();
}
```

```
function visualizaXML(tagArr){
    let txt= "";
    for (let x of tagArr){
        txt+= "<li>" + x.textContent + "</li>";
    }
    document.getElementsByTagName("ul")[0].innerHTML= txt;
}
```

```
▼<Universidad>
  ▼<UDC>
    <Año>1985</Año>
    <Rector>Enrique Pérez</Rector>
   ▼<Titulaciones>
      <Titulacion>Derecho</Titulacion>
      <Titulacion>Fisioterapia</Titulacion>
      <Titulacion>Ingeniería Informática</Titulacion>
    </Titulaciones>
  </UDC>
  ▼<USC>
    <Año>1992</Año>
    <Rector>Antonio Rodríguez</Rector>
   ▼<Titulaciones>
      <Titulacion>Farmacia</Titulacion>
      <Titulacion>Historia</Titulacion>
      <Titulacion>Ingeniería Informática</Titulacion>
      <Titulacion>Medicina</Titulacion>
    </Titulaciones>
  </USC>
  <UVigo>
    <Año>2005</Año>
    <Rector>José García</Rector>
   ▼<Titulaciones>
      <Titulacion>Ingeniería Informática</Titulacion>
      <Titulacion>Ingeniería de Telecomunicaciones</Titulacion>
      <Titulacion>Traducción e Interpretación</Titulacion>
    </Titulaciones>
  </UVigo>
</Universidad>
```

uniXMLFile.xml

ETSE

- Enrique Pérez
- Antonio Rodríguez
- José García

XMLHttpRequest

AJAX mediante el objeto *XMLHttpRequest*

```
<h2>ETSE</h2>
<ul></ul>
```

```
function cargaTexto(){
    let xhttp= new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            let jsonObj= JSON.parse(this.responseText);
            let tagArr= jsonObj.Universidad;

            visualizaJSON(tagArr);
        }
    };
    xhttp.open("GET", "uniJSONFile.json", true);
    xhttp.send();
}
```

```
function visualizaJSON(tagArr){
    let txt= "";
    for (let k in tagArr){
        txt+= "<li>" + tagArr[k].Rector + "</li>";
    }
    document.getElementsByTagName("ul")[0].innerHTML= txt;
}
```

```
{"Universidad":
    [{"Nombre": "UDC",
    "Año": "1985",
    "Rector": "Enrique Pérez",
    "Titulaciones": ["Derecho",
                    "Fisioterapia",
                    "Ingeniería Informática"]},
    {"Nombre": "USC",
    "Año": "1992",
    "Rector": "Antonio Rodríguez",
    "Titulaciones": ["Farmacia",
                    "Historia",
                    "Ingeniería Informática",
                    "Medicina"]},
    {"Nombre": "UVigo",
    "Año": "2005",
    "Rector": "José García",
    "Titulaciones": ["Ingeniería Informática",
                    "Ingeniería de Telecomunicaciones",
                    "Tradución e Interpretación"]}]
}
```

uniJSONFile.json

ETSE

- Enrique Pérez
- Antonio Rodríguez
- José García

JQuery

AJAX mediante *jQuery*

Utiliza el método load() para bajar datos desde el servidor e insertarlos en el objeto jQuery seleccionado.

.load(url, [data, function(response, status, xhr)])

- url → URL de los datos a cargar.
- data → Datos enviados al servidor junto a la petición.
- function(*response, status, xhr*):
  - response → Contiene los datos, resultado de la petición.
  - status → Contiene el estado, resultado de la petición.
  - xhr → Contiene el objeto *XMLHttpRequest.*

JQuery

AJAX mediante *jQuery*

Utiliza el método load() para bajar datos desde el servidor e insertarlos en el objeto jQuery seleccionado.

```
<h1>ETSE</h1>
<h2 id="demo"></h2>
```

```
AJAX USC
```

fileTXT.txt

ETSE

AJAX USC

```
<script>
$(document).ready(function(){
    $("h2").load("fileTXT.txt");
});
</script>
```

API Fech

AJAX mediante *API Fech*

Tradicionalmente *AJAX* se implementaba mediante el objeto *XMLHttpRequest*. Esta opción tenía un par de desventajas:

- Diferencias en la implantación entre fabricantes de navegadores distintos o versiones del mismo navegador.
- Trabaja con, *Callbacks* lo que tiende a crear un código más desorganizado

La solución tradicional pasaba por utilizar herramientas de terceros, como la librería *jQuery.*

Fech es una nueva interfaz estándar que permite el desarrollo del *frontend* basado en AJAX.

Utiliza *promesas* en lugar de *callbacks*, lo que facilita la organización del código asíncrono.

API Fech

AJAX mediante *API Fech*

El método fetch() → Método dependiente del objeto *window* del navegador.

`window.fetch(url)` → *`fetch(url)`* *(Forma abreviada de llamada)*

Fetch(url) → Traer al cliente el contenido de la *url,* devolviendo una *promesa* que resuelve en un objeto de la clase *Response* la petición, sea esta o no correcta.

API Fech

AJAX mediante *API Fech: Objeto Response*

Método	Acción
.text()	Devuelve una promesa con una representación en texto de los datos reccibidos
.json()	Devuelve una promesa con una representación en JSON de los datos reccibidos
.formData()	Devuelve una promesa con una representación en pares (clave, valor) de los datos recibidos
.bytes()	Devuelve una promesa con una representación en bytes de los datos recibidos

Propiedades	Valor
.status	Código de estado de la respuesta
.ok	Booleano indicando si la respuesta es exitosa
.headers	Objeto cabecera relativo a la respuesta
.url	URL de la respuesta

API Fech

AJAX mediante *API Fech*

```
let p= fetch('miUrl');
p.then(responseCallBack);
```

Llamada a fech() que devuelve un objeto promesa que se resuelve en un objeto de la clase Response

Objeto de la clase Response

```
function responseCallBack(r){
   let p1= r.text();
   p1.then(muestraTextoCallback);
}
```

Llamada al método text() que devuelve otro objeto promesa que se resuelve en una cadena de texto

Llamada a la función callback que muestra el contenido.

```
Function muestraTextoCallBack(txt){
          Código para mostrar el texto almacenado en txt
}
```

API Fech

AJAX mediante *API Fech*

El método fetch() → Método dependiente del objeto *window* del navegador.

`window.fetch(url)` → *`fetch(url)`* *(Forma abreviada de llamada)*

Fetch(url) → Traer al cliente el contenido de la *url,* devolviendo una promesa.

En forma compacta:

```
fetch('miUrl')
.then(r => r.text())
.then(txt => muestraTexto(txt));
```

Objetos Promesa

API Fech

AJAX mediante *API Fech*

```
<h1>ETSE</h1>
<h2 id="demo"></h2>
```

```
AJAX USC
```

fileTXT.txt

ETSE

AJAX USC

versión convencional

```
cargaTexto();

function cargaTexto() {
    let miP= fetch('fileTXT.txt')
    miP.then(ajaxOK);
}

function ajaxOK(r) {
    let miP1= r.text()
    miP1.then(muestraTexto);
}

function muestraTexto(txt) {
  document.getElementById("demo").innerHTML= txt;
}
```

versión arrow

```
cargaTexto();

function cargaTexto() {
  fetch('fileTXT.txt')
  .then(r => r.text())
  .then(txt => muestraTexto(txt));
}

function muestraTexto(txt) {
  document.getElementById("demo").innerHTML= txt;
}
```

API Fech

AJAX mediante *API Fech*

```
<h1>ETSE</h1>
<h2 id="demo"></h2>
```

```
{
  "Registros":[
  {"Nombre":"Pedro",
   "Ciudad":"Santiago",
   "Pais:":"España"},
  {"Nombre":"Pablo",
   "Ciudad":"León",
   "Pais:":"España"}
  ]
}
```

fileJSON.json

ETSE

- **Pedro -> Santiago**
- **Pablo -> León**

```
cargaJSON();

function cargaJSON() {
  fetch('fileJSON.json')
  .then(r => r.json())
  .then(jsonObj => muestraJSON(jsonObj));
}

function muestraJSON(jsonObj) {
    let txt='<ul>'
    for (x in jsonObj.Registros)
            txt= txt + "<li>" + jsonObj.Registros[x].Nombre + " -> "
                              + jsonObj.Registros[x].Ciudad + "</li>";
            txt= txt + "</ul>";
    muestraTexto(txt);
}

function muestraTexto(txt) {
  document.getElementById("demo").innerHTML= txt;
}
```

XMLHttpRequest

AJAX mediante *API Fech*

```
<h1>ETSE</h1>
<h2 id="demo"></h2>
```

```
cargaXML();

function cargaXML() {
    fetch('fileXML.xml')
    .then(r => r.text())
    .then(data => {
            const parser= new DOMParser();
            const xmlObj= parser.parseFromString(data,
                                                "application/xml");
            muestraXML(xmlObj);
      })
  }

  function muestraXML(xmlObj){

      let tag= xmlObj.getElementsByTagName("RECTOR");

      let txt='<ul>'
      for (let x of tag)
          txt+= "<li>" + x.textContent + "</li>";
      txt+= "</ul>";

      muestraTexto(txt);
  }

  function muestraTexto(txt) {
    document.getElementById("demo").innerHTML= txt;
  }
```

```
<?xml version="1.0" encoding="UTF-8"?>
<UNIVERSIDAD>
    <CORUÑA>
        <AÑO>1985</AÑO>
        <RECTOR>Enrique Pérez</RECTOR>
        <TITULACIONES>
            <TITULO>Derecho</TITULO>
            <TITULO>Fisioterapia</TITULO>
            <TITULO>Ingeniería Informática</TITULO>
        </TITULACIONES>
    </CORUÑA>
    <SANTIAGO>
        <AÑO>1992</AÑO>
        <RECTOR>Antonio Rodríguez</RECTOR>
        <TITULACIONES>
            <TITULO>Farmacia</TITULO>
            <TITULO>Historia</TITULO>
            <TITULO>Ingeniería Informática</TITULO>
            <TITULO>Medicina</TITULO>
        </TITULACIONES>
    </SANTIAGO>
    <VIGO>
        <AÑO>2005</AÑO>
        <RECTOR>Jose García</RECTOR>
        <TITULACIONES>
            <TITULO>Ingeniería Informática</TITULO>
            <TITULO>Ingeniería de Telecomunicaciones</TITULO>
            <TITULO>Traducción e Interpretación</TITULO>
        </TITULACIONES>
    </VIGO>
</UNIVERSIDAD>
```

fileXML.xml

ETSE

- **Enrique Pérez**
- **Antonio Rodríguez**
- **Jose García**

Desarrollo de Aplicaciones Web

AJAX

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces